

1. JavaFX

JavaFX es una biblioteca de Java que permite crear interfaces gráficas, como ventanas, botones, campos de texto y otros elementos visuales.

```
import javafx.application.Application;
import javafx.stage.Stage;
```

2. Application

Application es la clase base de una aplicación JavaFX. Se hereda para crear el programa y contiene el método **start()**, donde se construye la interfaz.

```
public class MiAplicacion extends Application {
    @Override
    public void start(Stage stage) {
        // Aquí se crea la interfaz
    }
}
```

3. Stage

Stage representa la **ventana principal** de la aplicación. En ella se muestra la interfaz gráfica.

```
public void start(Stage stage) {
    stage.setTitle("Mi aplicación");
    stage.show();
}
```

4. Scene

Scene representa el **contenido de una ventana (Stage)**. Dentro de ella se colocan los elementos de la interfaz.

```
Scene scene = new Scene(root, 400, 300);
stage.setScene(scene);
```

5. Node

Node es la clase base de los elementos visuales que forman parte de una **Scene**, como botones, etiquetas y campos de texto.

```
Button boton = new Button("Aceptar");
Label texto = new Label("Hola");
```

6. Controles

Los controles son elementos gráficos con los que el usuario puede **interactuar**, como **Button**, **TextField** y **Label**.

```
TextField nombre = new TextField();
Button boton = new Button("Mostrar");
Label resultado = new Label();
```

7. Layouts

Los layouts son contenedores que permiten **organizar y acomodar los elementos** de la interfaz. Por ejemplo, `VBox` coloca los elementos verticalmente.

```
VBox root = new VBox(10);

root.getChildren().addAll(
    nombre,
    boton,
    resultado
);
```

8. Eventos

Los eventos son acciones que ocurren en la aplicación, generalmente como respuesta a una acción del usuario, por ejemplo, hacer clic en un botón.

```
boton.setOnAction(e -> {
    resultado.setText("Hola");
});
```

9. `setOnAction()`

`setOnAction()` permite establecer **qué debe hacer un control cuando ocurre una acción**, principalmente cuando se presiona un botón.

```
boton.setOnAction(e -> {
    System.out.println("Botón presionado");
});
```

10. JavaFX y POO

JavaFX se relaciona con la POO porque sus elementos son **objetos creados a partir de clases**. Además, permite utilizar conceptos como herencia, encapsulamiento y polimorfismo.

```
public class Estudiante {
    private String nombre;

    public Estudiante(String nombre) {
        this.nombre = nombre;
    }

    public String getNombre() {
        return nombre;
    }
}
```

Aquí se utiliza **encapsulamiento**, porque `nombre` es `private` y se accede mediante `getNombre()`.

```
Estudiante estudiante = new Estudiante("Ingrid");
```

Aquí se crea un **objeto** de la clase `Estudiante`.